factor data type to character type before permanently converting the country data set into data frame.

```
#Convert from Factor to character
countries$Country.Region <-
as.character(countries$Country.Region)

#Convert to data frame
countries <- as.data.frame(countries)
```

Now, to combine countries and *map.world* data frames on the basis of region (*map.world*) and *Country.region* (countries) attributes, the *dplyr::left_join()* function is used.

```
# LEFT JOIN
map.world_joined <- left_join(map.
world, countries, by = c('region' =
'Country.Region'))
```

Figure 1 is a schematic diagram of the *left_join*, where Table 1 and Table 2 are the *map.world* and countries, respectively.

To reduce the mapping complexity here we shall mark only six countries, namely India, Nepal, Nigeria, Norway, US, Iran and regions of USA. This is done here with the filter *dplyr::semi_join()*. Figure 2 is a schematic diagram of the semi join between the attributes *countries (X)* and *df.country_points (Y)*.

```
df.country_points <- data.frame(country
= c("India","Nepal","Nigeria","Norway",
"US","Iran"), stringsAsFactors = F)

df.country_points <- semi_
join(countries, df.country_points, by =
c('Country.Region'='country' ))

glimpse(df.country_points)
```

So now we have two dataframes -- *map.world_joined* and *df.country_points*. Both are synchronised for the global map and six countries with latitudes and longitudes. The depiction of the world map and marking the map with COVID-19 infected data is shown here with *ggplot()*. With a little

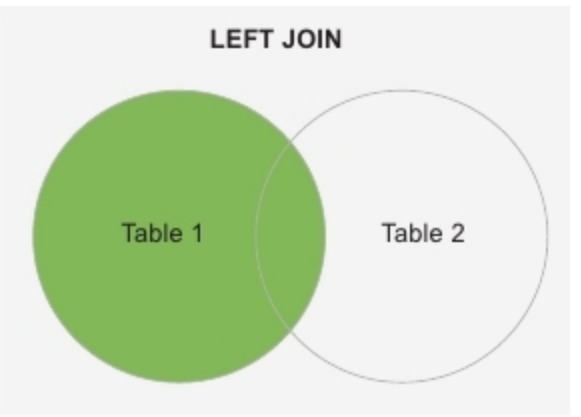

Figure 1: The left join between the variables *map.world* and countries

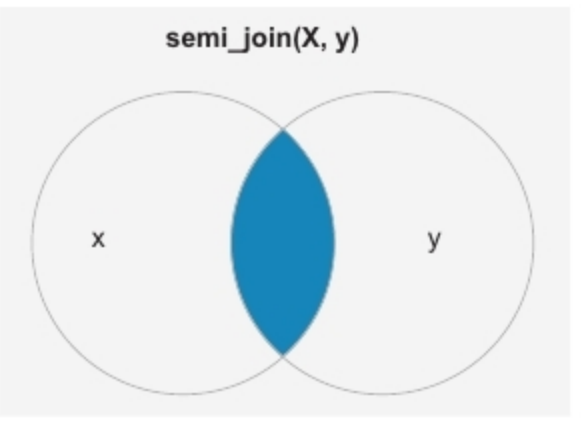

Figure 2: The semi join between the attributes countries and *df.country_points*.

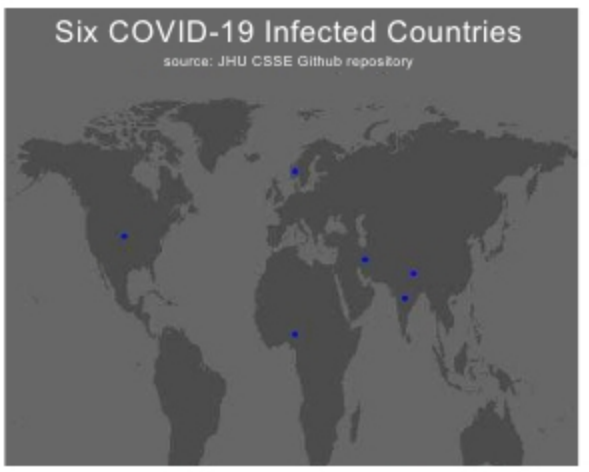

Figure 3: World map marked with India, Nepal, Nigeria, Norway, US, and Iran

patience, the reader can explore the following *ggplot()* command sequence:

```
p <-ggplot() +
  geom_polygon(data = map.world_joined,
aes(x = long, y = lat, group = group))
+
  geom_point(data = df.country_points,
aes(x = Long, y = Lat), color = "blue") +
  scale_fill_manual(values =
c("#CCCCCC","#e60000")) +
  labs(title = '             Six COVID-19
Infected Countries'
       ,subtitle = '
source: JHU CSSE Github repository') +
  theme(text = element_text(family =
"Verdana", color = "#FFFFFF")
        ,panel.background = element_
rect(fill = "#444444")
        ,plot.background = element_
rect(fill = "#444444")
   ,panel.grid = element_blank()
   ,plot.title = element_text(size = 20)
        ,plot.subtitle = element_
text(size = 10)
        ,axis.text = element_blank()
        ,axis.title = element_blank()
        ,axis.ticks = element_blank()
        ,legend.position = "none"
        )
library(gridExtra)
grid.arrange(p, ncol = 1)

closeAllConnections()
```

Due to the easy availability of different map packages, this feature of R has become a useful tool for practising data visualisation. There is lots of data available in many places that data analytics professionals can map.

Creating maps typically requires several essential skills in combination. These include the ability to retrieve the data, reshape it into the proper format, filtering and merging with other data and visualising it. We hope this article will help the reader to learn the visualisation techniques of R. COVID-19 data is available in different repositories, so the readers interested in it can download the data and explore it for further studies. END

By: Dr Dipankar Ray

The author is a member of IEEE, IET (UK) and has more than 20 years of experience in open source versions of UNIX operating systems and SUN Solaris, along with experience in programming and project development. He is presently working on data analysis and machine learning using neural networks and different statistical tools.